THE UNIVERSITY OF MEMPHIS

Compiling and Running GPU-Accelerated Workloads on BigBlue

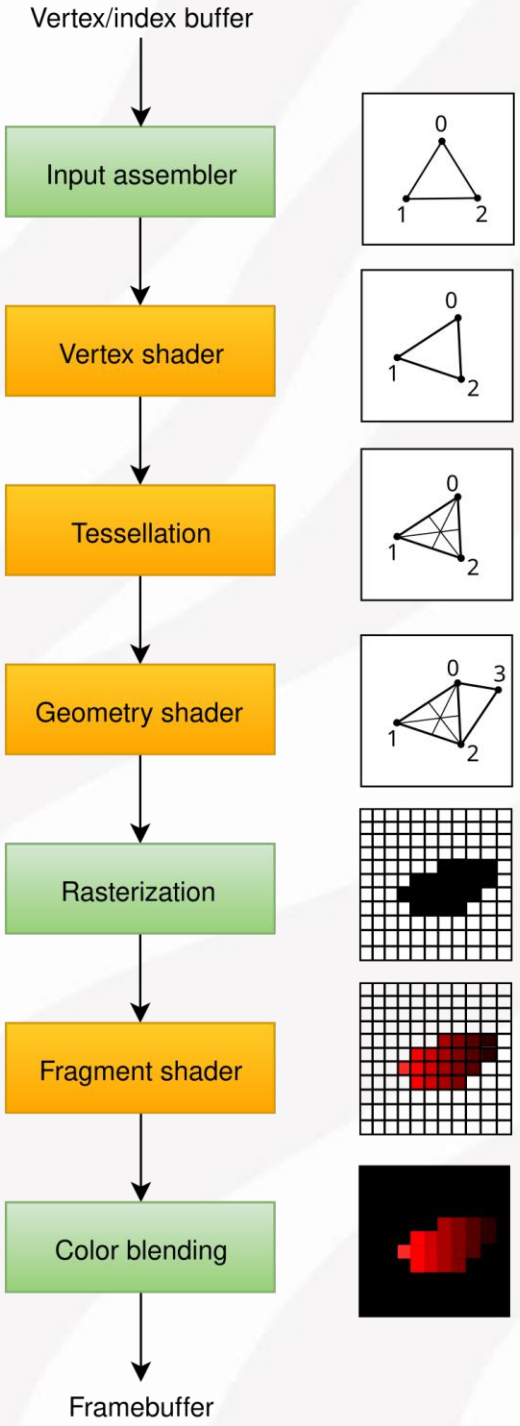
What is a GPU?

Graphics Processing Unit
Computes:
- Geometry
- Color
- Texture

Advantages:
- Highly parallel
- Simple to program

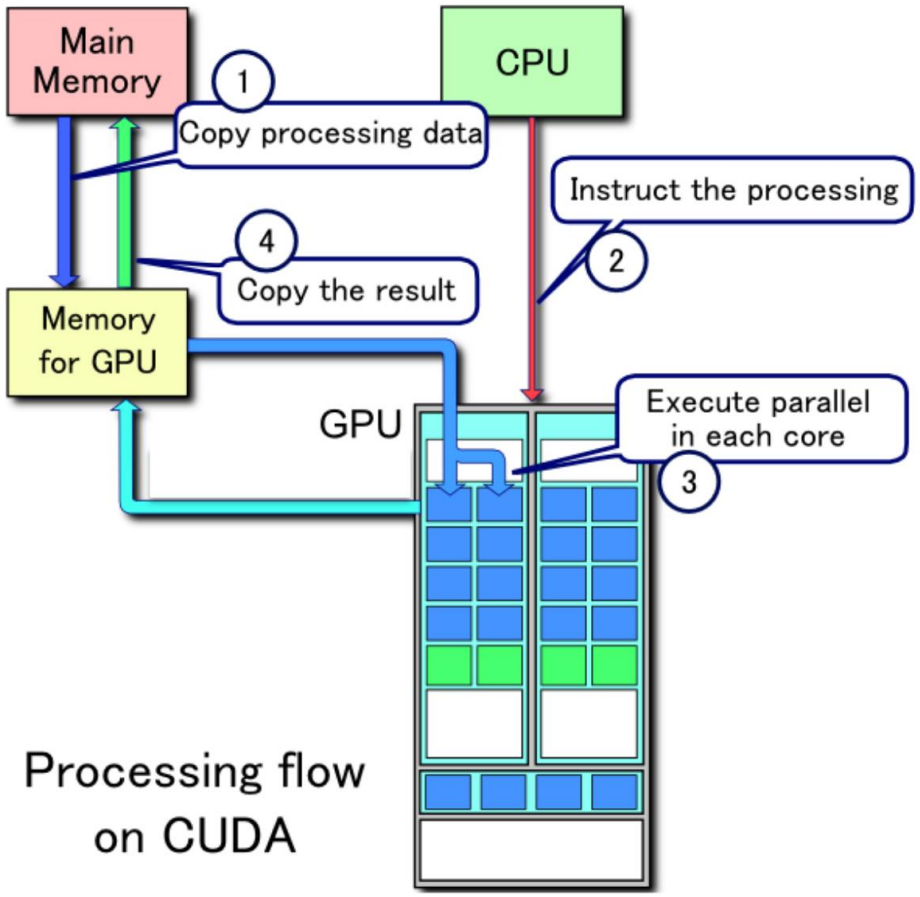
Disadvantages:
- Fixed function pipeline
- Usually does not send data back to host memory

https://commons.wikimedia.org/wiki/File:Evgageforce8800gtxtopleft.jpg



https://en.wikipedia.org/wiki/Shader

What is GPGPU?



https://commons.wikimedia.org/wiki/File:CUDA_processing_flow_(En).PNG

General Purpose Computation on Graphics Processing Unit

- All the advantages of parallel computation on GPU
- Can send the data back to host memory

Disadvantages

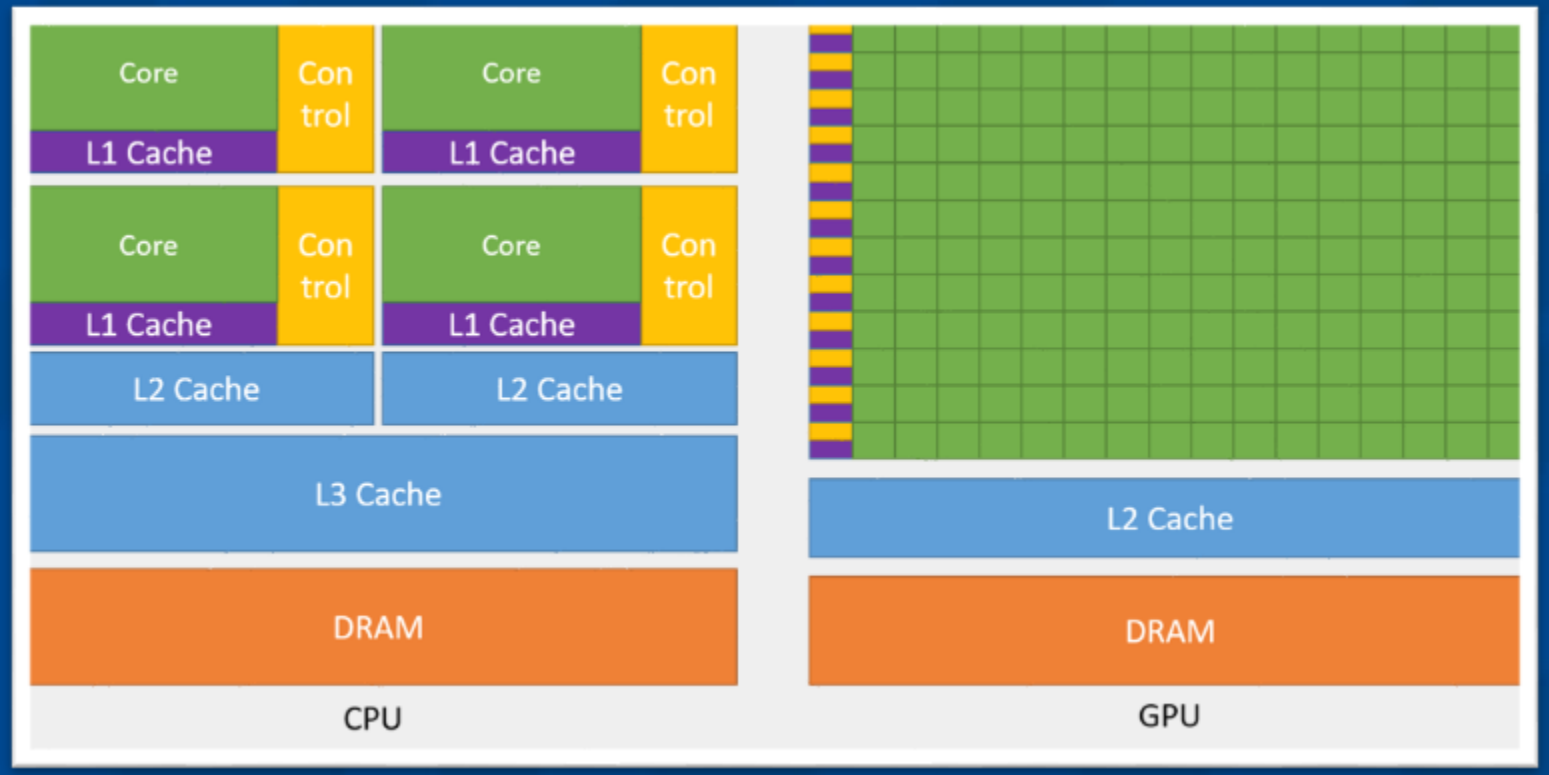
- Still a GPU with a fixed function pipeline, although mitigated with unified shader processors
- Transfers are bottlenecked by the communication bus between GPU and CPU

- Textures = Arrays
- GPU programs = Kernel programs
- Rendering = Program execution

https://www.cs.york.ac.uk/rts/docs/GECCO_2007/docs/p1566.pdf

GPU vs CPU

How different are they?



https://docs.nvidia.com/cuda/cuda-programming-guide/01-introduction/introduction.html

THE UNIVERSITY OF MEMPHIS

THE UNIVERSITY OF MEMPHIS

GPU vs CPU

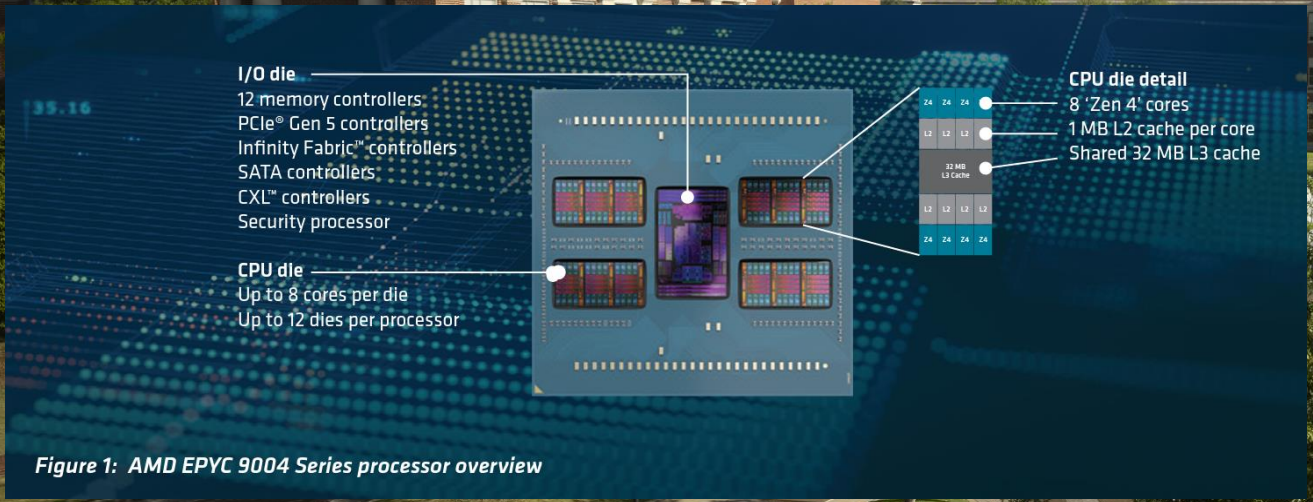
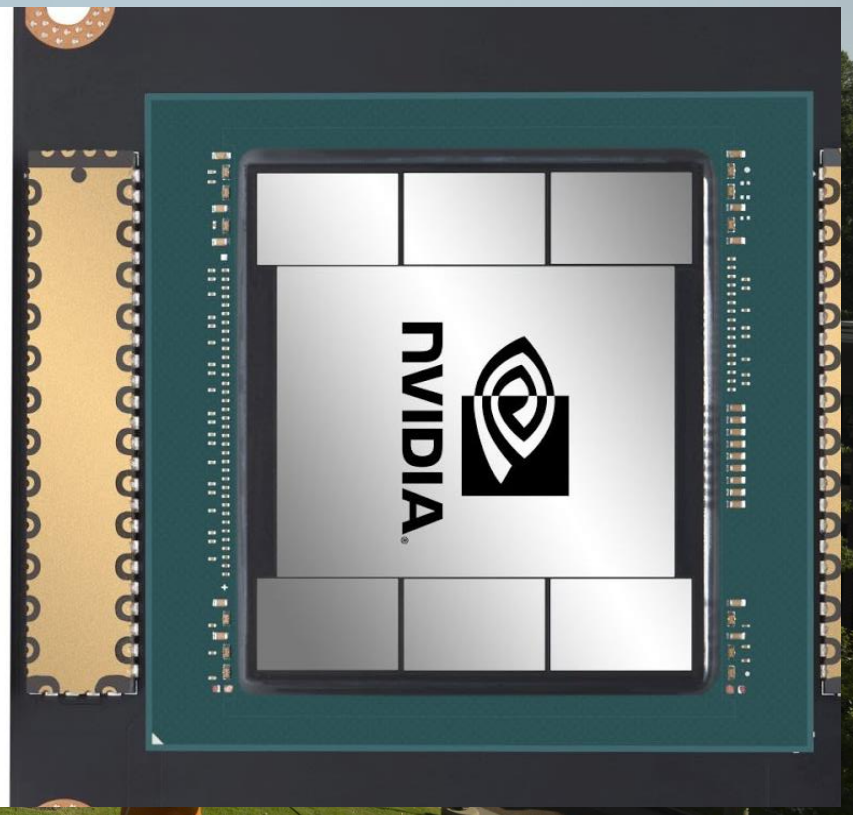


Figure 1: AMD EPYC 9004 Series processor overview



CPU

- Multiple instructions, multiple data
- Complex threading
- Large caches
- Complex input/output systems
- Low latency
- Medium throughput
- High precision
- Complex memory hierarchy

GPU

- Single instructions, multiple data/threads
- Simple threading
- Smaller caches
- Very Simple input/output systems
- High latency
- High throughput
- Low to moderate precision
- Simple memory hierarchy

4th Gen AMD EPYC 9654 CPU theoretical performance would be:

96(# of cores) x **2.4 GHz**(base clock speed in GHz) x**16**(AMD EPYC 9004 floating-point operations per clock cycle multiplier)**= 3.6864 TFLOPS.**

https://www.amd.com/en/blogs/2025/leadership-hpc-performance-with-5th-generation-amd.html

7.3728 FP64 TFlops for 2 CPUs

24 lanes of DDR5 4800 memory
921.6 GB/s memory bandwidth

https://www.amd.com/en/products/processors/server/epyc/4th-generation-9004-and-8004-series/amd-epyc-9654.html

	A100 80GB PCIe
FP64	9.7 TFLOPS
FP64 Tensor Core	19.5 TFLOPS
FP32	19.5 TFLOPS
Tensor Float 32 (TF32)	156 TFLOPS | 312 TFLOPS*
BFLOAT16 Tensor Core	312 TFLOPS | 624 TFLOPS*
FP16 Tensor Core	312 TFLOPS | 624 TFLOPS*
INT8 Tensor Core	624 TOPS | 1248 TOPS*

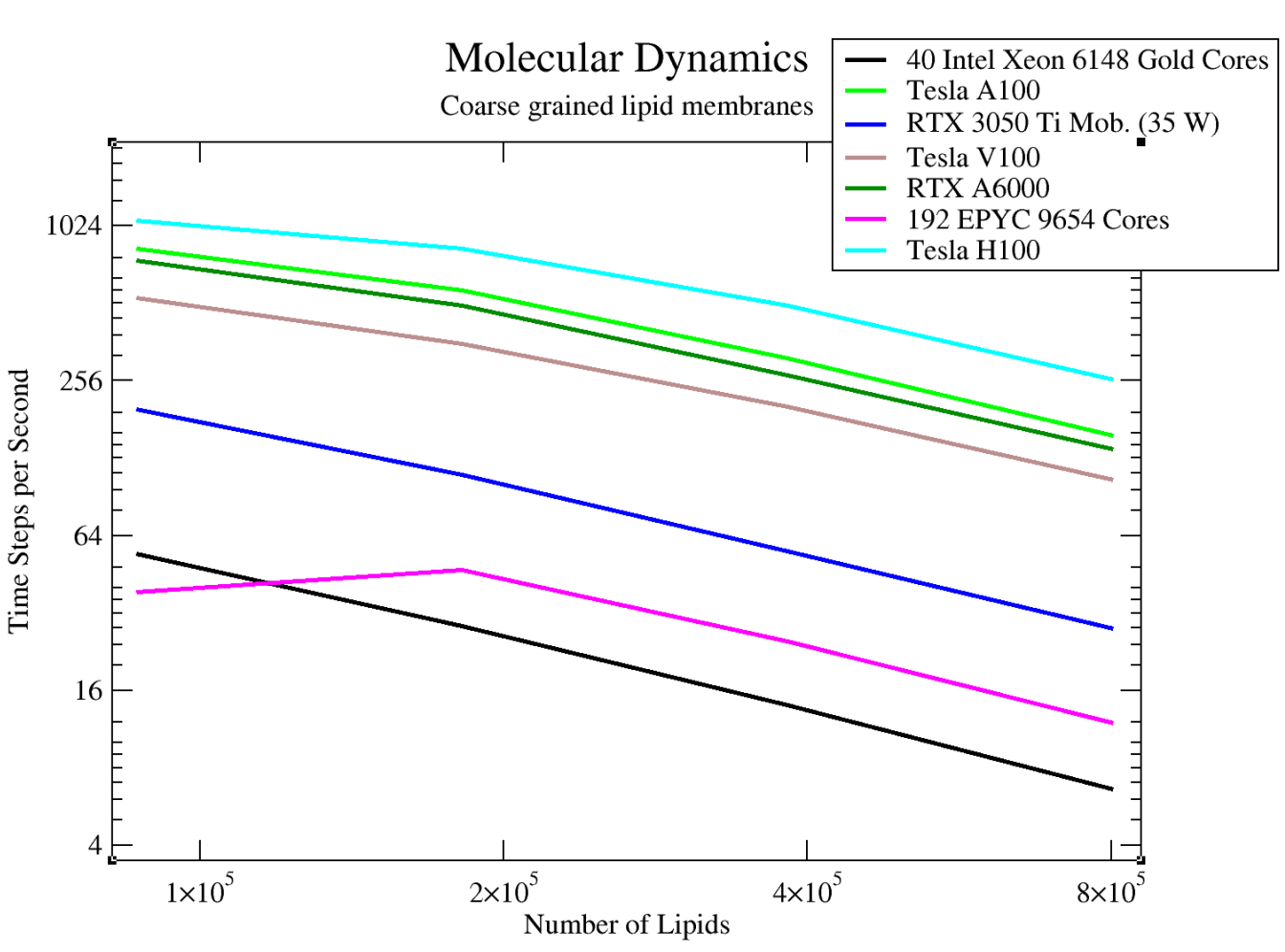
19.4 FP64 TFlops for 2 GPUs

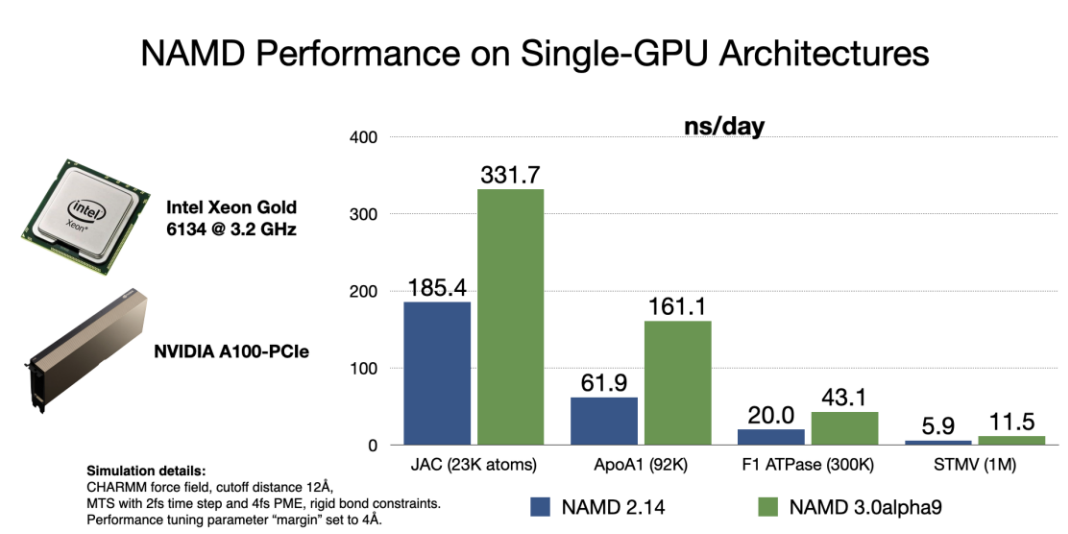
3870 GB/s memory bandwidth

https://www.nvidia.com/en-us/data-center/a100/

https://github.com/uofm-research-computing/hpl-tests/tree/main

What about real benchmarks?



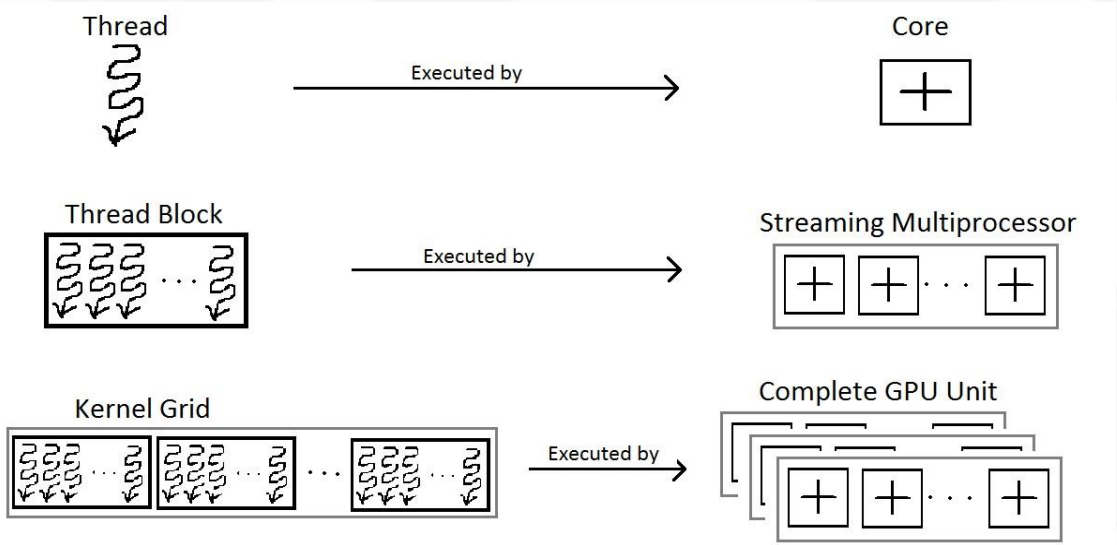


https://www.ks.uiuc.edu/Research/namd/benchmarks/NAMD_performance.001.png

GPU "Kernel"

- Small code segment that is executed on every GPU core, identically, as a "thread"
- Each "thread" is assigned a set of indices in a "block" on a "grid"
- Processing is controlled by a GPU schedular
- Can be written in
  - CUDA - "**cuda**" and "**nvhpc**" modules
  - OpenCL - "cuda" module and compile OpenCL header files
  - SyCl - AdaptiveCPP compilation required with "cuda" module, or intel oneAPI and "cuda" module
  - OpenMP/OpenACC - "**nvhpc**" module, usable with C/C++ and Fortran
  - PyCUDA/NUMBA - python installation, "**python/3.12.9**" module
  - Julia - Can also just use CUDA in julia

https://x-dev.pages.jsc.fz-juelich.de/models/



https://en.wikipedia.org/wiki/Thread_block_(CUDA_programming)

```
__global__ void vecAdd(float* A, float* B, float* C, int vectorLength)
{
    // calculate which element this thread is responsible for computing
    int workIndex = threadIdx.x + blockDim.x * blockIdx.x

    if(workIndex < vectorLength)
    {
        // Perform computation
        C[workIndex] = A[workIndex] + B[workIndex];
    }
}
```

https://docs.nvidia.com/cuda/cuda-programming-guide/02-basics/intro-to-cuda-cpp.html

CUDA Accelerated Libraries

PTX

Kernels are converted to a PTX format for GPU Driver

```
__global__ void smem_cuda_transpose(int m,
                                    float *a,
                                    float *c )
{

    /* declare a statically allocated shared memory array */

    __shared__ float smemArray[THREADS_PER_BLOCK_X][THREADS_PER_BLOCK_Y];

    /* determine my row and column indices for the error checking code */

    const int myRow = blockDim.x * blockIdx.x + threadIdx.x;
    const int myCol = blockDim.y * blockIdx.y + threadIdx.y;

    /* determine my row tile and column tile index */

    const int tileX = blockDim.x * blockIdx.x;
    const int tileY = blockDim.y * blockIdx.y;
```

```
.visible .entry nvkernel_main_F1L38_2(
        .param .u64 nvkernel_main_F1L38_2_param_0,
        .param .u64 nvkernel_main_F1L38_2_param_1,
        .param .u64 nvkernel_main_F1L38_2_param_2,
        .param .u64 nvkernel_main_F1L38_2_param_3
)
.maxntid 128, 1, 1
{
        .reg .pred      %p<13>;
        .reg .f32       %f<8>;
        .reg .b32       %r<72>;
        .reg .b64       %rd<52>;
        .loc    1 38 0


        ld.param.u64    %rd5, [nvkernel_main_F1L38_2_param_2];
        ld.param.u64    %rd6, [nvkernel_main_F1L38_2_param_3];
        ld.param.u32    %r1, [nvkernel_main_F1L38_2_param_0];
        ld.param.u32    %r2, [nvkernel_main_F1L38_2_param_1];
        .loc    1 40 1
        add.s32         %r3, %r1, -1;
        setp.lt.s32     %p1, %r3, 0;
        setp.lt.s32     %p2, %r1, 1;
        or.pred         %p3, %p2, %p1;
        @%p3 bra        $L__BB0_13;
```

https://docs.nvidia.com/cuda/parallel-thread-execution/index.html

BigBlue

"igpuq" partition

- 2 Nvidia V100 GPUs per node
- 16 GB of device memory
- PCI-express bus

- 2 Intel Xeon 6148 Gold CPUs
- 192 GB of host memory

Tesla V100 for PCIe
DOUBLE-PRECISION 7 teraFLOPS
SINGLE-PRECISION 14 teraFLOPS
DEEP LEARNING 112 teraFLOPS

https://www.nvidia.com/en-gb/data-center/tesla-v100/

"agpuq" partition

- 2 Nvidia A100GPUs per node
- 80 GB of device memory
- PCI-express bus

- 2 AMD EPYC 9654 CPUs
- 768 GB of host memory

	A100 80GB PCIe
FP64	9.7 TFLOPS
FP64 Tensor Core	19.5 TFLOPS
FP32	19.5 TFLOPS
Tensor Float 32 (TF32)	156 TFLOPS | 312 TFLOPS*
BFLOAT16 Tensor Core	312 TFLOPS | 624 TFLOPS*
FP16 Tensor Core	312 TFLOPS | 624 TFLOPS*
INT8 Tensor Core	624 TOPS | 1248 TOPS*

https://www.nvidia.com/en-us/data-center/a100/

Slurm options

"sbatch", "srun", and "salloc"

- Number of GPUs: "--gres=gpu:N"
  - "N" Per node, up to 2
- Partition: "--partition=igpuq,agpuq"
  - Can select one or more partitions
- Number of GPUs: "--gpus=gpu:N"
  - Total "N" for job
- Number of GPUs per task: "--gpus-per-task=gpu:N"
  - MPI tasks

Note: No need to set "CUDA_VISIBLE_DEVICES" if you run a job on the cluster!

Nvidia-smi interactive job

Selecting 1 GPU

```
[jspngler@log02 ~]$ srun -c 1 -p igpuq --gres=gpu:1 nvidia-smi
srun: job 2740135 queued and waiting for resources
srun: job 2740135 has been allocated resources
cpu-bind=MASK - ig01, task  0  0 [2370681]: mask 0x20 set
Thu Mar 26 02:14:50 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 545.23.08              Driver Version: 545.23.08    CUDA Version: 12.3     |
|-----------------------------------------+----------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |         Memory-Usage | GPU-Util  Compute M. |
|                                         |                      |               MIG M. |
|=========================================+======================+======================|
|   0  Tesla V100-PCIE-16GB           On  | 00000000:3B:00.0 Off |                    0 |
| N/A   29C    P0              24W / 250W |      0MiB / 16384MiB |      0%      Default |
|                                         |                      |                  N/A |
+-----------------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                          GPU Memory |
|        ID   ID                                                           Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

Selecting 2 GPUs

```
[jspngler@log02 ~]$ srun -c 1 -p igpuq --gres=gpu:2 nvidia-smi
srun: job 2740136 queued and waiting for resources
srun: job 2740136 has been allocated resources
cpu-bind=MASK - ig04, task  0  0 [6399]: mask 0x1 set
Thu Mar 26 02:14:58 2026
+-----------------------------------------------------------------------------------------+
| NVIDIA-SMI 545.23.08              Driver Version: 545.23.08    CUDA Version: 12.3     |
|-----------------------------------------+----------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |         Memory-Usage | GPU-Util  Compute M. |
|                                         |                      |               MIG M. |
|=========================================+======================+======================|
|   0  Tesla V100-PCIE-16GB           On  | 00000000:3B:00.0 Off |                    0 |
| N/A   26C    P0              24W / 250W |      0MiB / 16384MiB |      0%      Default |
|                                         |                      |                  N/A |
+-----------------------------------------+----------------------+----------------------+
|   1  Tesla V100-PCIE-16GB           On  | 00000000:D8:00.0 Off |                    0 |
| N/A   27C    P0              25W / 250W |      0MiB / 16384MiB |      0%      Default |
|                                         |                      |                  N/A |
+-----------------------------------------+----------------------+----------------------+

+-----------------------------------------------------------------------------------------+
| Processes:                                                                              |
|  GPU   GI   CI        PID   Type   Process name                          GPU Memory |
|        ID   ID                                                           Usage      |
|=========================================================================================|
|  No running processes found                                                             |
+-----------------------------------------------------------------------------------------+
```

Guiding the compiler

Several methods to compile projects:

- Directly compiling
- Using a build tool like cmake or make
- Specifying the packages using python or julia

Direct Compilation

nvcc using default architecture (5.0 from 2014):

```
module load cuda
nvcc input.cu -o output
```

nvcc with more optimization (compute 7.0 and 8.0 from 2017 and 2020, respectively):

```
module load cuda
nvcc input.cu -o output -arch=compute_70 --gpu-code sm_70,sm_80
```

nvcc on itiger (compute 8.6 and 9.0 from 2022):

```
wget https://developer.download.nvidia.com/compute/cuda/12.6.3/local_installers/cuda_12.6.3_560.35.05_linux.run
bash cuda_12.6.3_560.35.05_linux.run --toolkit --toolkitpath=/home/$USER/.local/cuda --silent
#These next two lines could be in a script or ~/.bashrc
export PATH=/home/$USER/.local/cuda/bin:$PATH
export LD_LIBRARY_PATH=/home/$USER/.local/cuda/lib64:$LD_LIBRARY_PATH
nvcc input.cu -o output -arch=compute_86 --gpu-code sm_86,sm_90
```

nvc++ (also nvc and nvfortran):

```
module load nvhpc
nvc++ input.cpp -o output -O3 -fopenmp -mp=gpu -gpu=cc70,cc80,managed,cuda12.3
```

Direct C

Feature support (unlisted features are supported for all compute capabilities)	Compute capability (version)								
	1.0, 1.1	1.2, 1.3	2.x	3.0	3.2	3.5, 3.7, 5.x, 6.x, 7.0, 7.2	7.5	8.x	9.0, 10.x, 12.x
Warp vote functions (__all(), __any())	No	Yes							
Warp vote functions (__ballot())	No		Yes						
Memory fence functions (__threadfence_system())									
Synchronization functions (__syncthreads_count(), __syncthreads_and(), __syncthreads_or())									
Surface functions									
3D grid of thread blocks									
Warp shuffle functions	No			Yes					
Unified memory programming									
Funnel shift	No				Yes				
Dynamic parallelism	No					Yes			
Uniform Datapath[65]	No						Yes		
Hardware-accelerated async-copy	No							Yes	
Hardware-accelerated *split arrive/wait barrier*									
Warp-level support for reduction ops									
L2 cache residency management									
DPX instructions for accelerated dynamic programming	No								Yes
Distributed shared memory									
Thread block cluster									
Tensor memory accelerator (TMA) unit									
Feature support (unlisted features are supported for all compute capabilities)	1.0, 1.1	1.2, 1.3	2.x	3.0	3.2	3.5, 3.7, 5.x, 6.x, 7.0, 7.2	7.5	8.x	9.0, 10.x, 12.x
	Compute capability (version)								

https://en.wikipedia.org/wiki/CUDA

Make

Check the software's documentation! Sometimes you need to modify the actual makefile

Common pattern:

```
module load cuda
./configure --prefix=/home/$USER/.local
make install
```

Common makefile variables related to GPUs:

- LD_LIBRARY_PATH – where shared libraries are located, commonly set after an application is compiled, typically "lib*.so" files
- PATH – where executables are located, commonly set after an application is compiled
- CFLAGS, CXXFLAGS, FFLAGS – compilation flags for optimization and debugging
- LDFLAGS – linker flags for libraries, commonly "-lcuda", "-lcudart" with Nvidia GPUs

Libraries that use this system:

- Most linux software
- Researchers (it is really easy to setup)

cmake

Check the software's documentation!

Common pattern:

```
module load cuda
mkdir build && cd build
cmake .. -DCMAKE_INSTALL_PREFIX=/home/$USER/.local -DCMAKE_CUDA_ARCHITECTURES="70;80"
cmake --build .
cmake --install .
```

Common cmake variables related to GPUs:

- CMAKE_CUDA_ARCHITECTURE – Sets the GPU compute capability

Note on cmake 3.17:
Find_package(CUDA) was common before this version. It looks for the driver, sdk, and toolkit.
Find_package(CUDAToolkit) fixes some issues by only looking for the toolkit
You can typically swap these out to fix compilation issues, and contact the maintainers to remind them to update this

Many people use this for windows/linux/mac cross compatibility

https://cmake.org/cmake/help/latest/module/FindCUDAToolkit.html

python

Check the software's documentation!

Common pattern:

```
module load cuda python/3.12.9
Pip3 install package
```

Or:

```
module load cuda python/3.12.9
python setup.py install
```

Common python variables related to GPUs:

- Usually none, but sometimes packages specify cuda versions.
  - For example: pip3 install cudf-cu12
- Typically binary distributions, but some packages compile components
- Make sure to load the cuda module if you use GPUs, but many packages will also download a version of the cuda toolkit

Conclusions

- GPUs can speed-up your workloads
- Prefer using packages that can be compiled over prebuilt ones
- Lower precision workloads are significantly faster on GPUs

We have several examples on github:
https://github.com/orgs/uofm-research-computing/repositories

THE UNIVERSITY OF MEMPHIS